

Project Requirements Document

Team: Moose Blazers

Project: The Aerospace Capstone

Team: Jordan Pringle, Melissa Anewalt, Peter Gaede, Scott Walstead

Revision: 2

Intro:

In our capstone project we will showcase the low-power, high-performance processing of NVidia's Jetson TK1 board utilizing the advantages of GPGPU parallel processing (CPU+GPU) over serial (CPU) processing. A GUI will allow a user to choose among various computer vision applications such as video compression/quality and motion tracking parameters. Finally the application will display performance data comparing serial and parallel versions of the same application.

Glossary of Terms:

-GPGPU: General-Purpose computing on Graphics Processing Units where a computer's GPU handles some of the tasks alongside the CPU.

-Computer Vision: A field of study focusing on the processing and analysis of images and video on a computer to understand the scenes represented in them.

-CUDA: A parallel processing programming model created by NVidia to run programs on their lines of graphics processors.

System Architecture Overview:

Our applications will run onboard the NVidia Jetson TK1 development board. The user can interact with the application via a connected keyboard/mouse and monitor or via SSH remote login. The user will browse for an input video file or choose to stream from a webcam connected to the board, and can select between different options of computer vision analysis on the input such as object detection, image compression, color conversion, or edge detection. From the selected video file or webcam input, the program will run an edge detection algorithm with thresholding to find a desired object in the scene. With that thresholded image, the program then finds the center of the detected object to calculate its speed from frame to frame. If the speed is slow or no object is detected, a compression algorithm will heavily compress the image to save the most space on the disk and save the most energy in a theoretical transmission of that output. When the object is reintroduced into the scene or it speeds up, the compression algorithm will adjust to allow more detail so more scientific data could be collected from the output. The output video stream will play in real-time as analysis overlaid on the input file, which can be saved to a video file.

Requirements:

1. As a developer, I can understand CUDA so that I can write both serial (CPU) and parallel (CPU+GPU) versions of computer vision applications.

2. As a developer, I can understand OpenCV so that I can write both serial (CPU) and parallel (CPU+GPU) versions of computer vision applications.
3. As an end user, I can run a serial (CPU) version of an application and I can view its performance metric.
4. As an end user, I can run a parallel (GPU) version of an application and I can view its performance metric.
5. As an end user, I can use the "G" key on the keyboard to toggle between grayscale and color output.
6. As a continuity checking application, the outputs of the serial/parallel versions of the same application are checked to ensure both give the same results.
7. As a motion tracking application, the activity of tracked objects can be measured so that the resolution and compression of a video stream can be adjusted accordingly.
8. As a motion tracking application, the resolution and data sampling rate will increase when activity in the scene passes above a threshold in order to display interesting information in detail.
9. As a motion tracking application, the resolution and data sampling rate will decrease when activity in the scene passes below a threshold in order to conserve power when nothing interesting is happening.
10. As a performance metric, the performance of both versions will be displayed graphically and textually.
11. As a developer, I can change the frequency of the Jetson TK1 board so that I can find the power used by the whole system.
12. As a developer, I can write a program that serially uses a binary threshold so that I can make a black and white image.
13. As a developer, I can write a binary threshold program that uses parallel programming so that I can make a black and white image.
14. As a developer, I can write a program that serially finds the center of an object so that I can find the velocity of the object.
15. As a developer, I can write a parallel program that finds the center of an object so that I can find the velocity of the object.
16. As a developer, I can write a parallel Gaussian Filter in order to smooth an image.
17. As a developer, I can write a serial Gaussian Filter in order to smooth an image.
18. As a developer, I can serially find the intensity gradients of an image so that I can find horizontal, vertical, and diagonal edges in an image.
19. As a developer, I can use parallelism to find the intensity gradients of an image so that I can find horizontal, vertical, and diagonal edges in an image.
20. As a developer, I can serially use Non-Maximum Suppression in order to thin the edges.
21. As a developer, I can use Non-Maximum Suppression in parallel in order to thin the edges.
22. As a developer, I can serially use a double threshold so that I can find potential edges of an image.

23. As a developer, I can use parallelism to use a double threshold so that I can find potential edges of an image.
24. As a developer, I can serially track edges by using hysteresis so that I can find strong edges.
25. As a developer, I can use parallelism to track edges by using hysteresis so that I can find strong edges.
26. As an end user, I can use a graphical file system so that I can choose an input file.
27. As an end user, I can use an attached webcam so that I can run programs off its input.
28. As a GUI, I have a clickable button to open a graphical file system so that a user can pick an input file.
29. As a GUI, I have a clickable button to activate an attached webcam so that a user can use its output in a function.
30. As an end user, I can use the keyboard so that I can make changes to my serial/parallel selections on a running application.

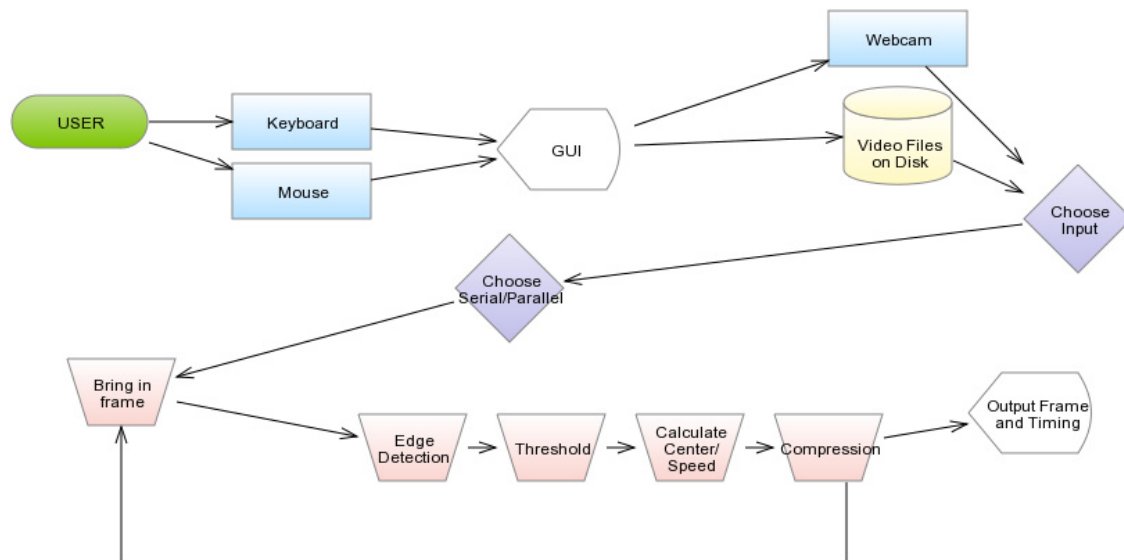
Test Cases:

<https://github.com/scotty914/GPGPU-Aerospace/>

1. Be able to write a parallelized function using CUDA.
2. Be able to write a parallelized function using OpenCV.
3. Be able to select serial processing or parallel processing from a drop down menu
4. Be able to select a video file and load it into the GUI
5. Does the press of "G" get passed to the program? Does the program toggle between grayscale and color with a press of "G"?
6. Does the test application that compares the outputs of the CPU and GPU applications return with 0? If they do not return 0, return error message.
7. Does the application display the speed of a tracked object? Is the displayed speed accurate?
8. When the activity in the scene is above a set threshold, does the resolution increase to a high set value?
9. When the activity in the scene is below a set threshold, does the resolution decrease to a low set value?
10. Does the conclusion of an application bring up a performance statistics page? Are the performance statistics relevant to the application that just ran?
11. When checking the Jetson TK1 board, is the frequency set to what was programmed.
12. Will my program take an image and change the pixels according to the threshold value?
13. Will my program go through each pixel in a parallel manner and change the pixels according to the threshold value?
14. Does my program find the center of an object?
15. Does my program use parallelism to find the center of an object?
16. Does my code use parallelism to apply a Gaussian Filter to blur an image?
17. Does my code apply a Gaussian Filter to blur an image?
18. Can I find the gradient intensities? Are they correct?

19. Can I find the gradient intensities using parallelism? Are they correct?
20. Are edges thinned out when calling on the Non-Maximum Suppression?
21. Are edges thinned out when calling on the Non-Maximum Suppression by using parallelism?
22. Does the double threshold I use get rid of extra noise?
23. Does the double threshold I use get rid of extra noise by using parallelism?
24. Does the hysteresis correctly get rid of weak edges?
25. Does the hysteresis correctly get rid of weak edges using parallelism?
26. Can I traverse through the file system to choose an image?
27. Is the webcam setup and can it display the live stream? If the webcam is not setup does it return an error?
28. Can I click on a button so that the graphical file system is displayed?
29. If I click on the button that brings me to the webcam? If the webcam is not attached does an error message appear?
30. Does my keyboard accurately transfer between serial and parallel. If I push a wrong button does an error message appear?

System Models:



Technologies:

Hardware:

- NVidia Jetson TK1 for running code
- Creative Live! Cam Chat HD 720p WebCam over USB 3.0
- Keyboard/Mouse for User Input
- Display for output
- 64GB SanDisk Card for media storage

Software/Libraries:

- OpenCV4Tegra for L4T 21.x for processing images and videos
- CUDA 6.5 for parallelizing code on the GPU
- GTK+ 2.0 for the initial GUI that allows user to choose file or webcam input
- NCurses for the command line UI that allows user to toggle options

dynamically

- Linux For Tegra R21.2 derived from Ubuntu 14.04 LTS